EEE3096S Practical 3 Report

Roston Smith
SMTROS022
Github Link: https://github.com/RostonSmith/EEE3096S-2025-Group_98.git

***Abstract* - This report documents the porting, benchmarking and analysis of a Mandelbrot renderer executed on STM32F0 and STM32F4 microcontrollers. The experiments evaluate the influence of maximum iteration counts, compiler optimization levels, floating-point unit (FPU) availability and fixed-point scaling on performance (execution time, CPU cycles, pixels/sec) and accuracy. The aim is to quantify throughput and trade-offs for embedded numerical workloads. (Keywords—Mandelbrot, STM32F4, STM32F0, profiling, DWT cycle counter)**

I. Introduction (*Heading 1*)

This practical (EEE3096S Practical 3) requires porting the Mandelbrot implementation from earlier practicals to an STM32F4 board and benchmarking it alongside the STM32F0 implementation. Objectives include measuring execution time, CPU cycles, throughput (pixels/sec), accuracy (checksums), and assessing the effects of MAX_ITER, FPU, floating vs double precision, compiler optimizations, and fixed-point scaling. The practical spec and deliverables are followed closely.

II. Tasks & Objectives

Task 1: Port Mandelbrot to STM32F4 and log baseline performance (execution time, checksum).

Task 2: Test 5 MAX_ITER values (e.g., 100, 250, 500, 750, 1000) on both F0 and F4 and record execution time and checksum.

Task 3: Record CPU cycles (using DWT), wall-clock time and compute throughput (pixels/sec), constraint MAX_ITER=100.

Task 4: Scalability: increase image size up to 1920×1080 (Full HD), document splitting strategy if memory constrained.

Task 5: FPU impact — run float vs double, FPU enabled vs disabled on STM32F4; report accuracy & speed-up.

Task 6: Compiler optimizations — test at least three optimization levels (e.g., -O0, -O2, -Os). Record binary file size and runtime.

Task 7: Fixed-point arithmetic — test at least three scaling factors and report precision, overflow risk, and speed.

Task 8: Power measurement attempt — describe method and results or explain constraints and required tools.

III. Methodology

The methodology followed a structured implementation of Assembly routines to meet the given tasks. The steps undertaken were:

A. Optimising Prac 1B Code and Porting to F4 Platform

By utilising an array for the image size and using a for loop to iterate through the image values and storing the results in their own arrays, one can simply just run the benchmark in debugging mode and leave the program to run fully, after which the values can be recorded. This coupled with individual copies of the main.c files for each task and for each board that can be edited in isolation, allow for optimal use of time while the length benchmark runs.

B. *Adding a Second Array for MAX_ITERs*

By adding the second array for max_iter values, this follows the time efficiency of Task . Using a nested for loop and storing the results in 2D arrays allows for all results for each board to be recorded while other tasks are performed.

C. *Advancing Timing*

By implementing the use of DWT where it is available on the F4 platform, a more accurate time reading is available, and the use of CPU cycles and throughput allow for a more in-depth comparison of the MCUs. These are pretty easy to implement with the available functions for getting cpu_cycles and then using the known clock frequency to calculate the execution time in seconds and using that for the throughput.

D. *Scalability Testing*

By implementing a method to break down the image sizes in to more manageable tiles for the MCU to process, RAM usage can be decreased significantly. For the F4 platform, it was also viable to implement Adaptive Tiling, which uses larger tiles as the images get larger. This ensures that the number of tiles doesn't get too large as this can cause the program to slow down as tile are computed sequentially rather than in parallel. The repeated calculation of small tiles proves expensive when factoring in a small delay in each calculation to stabilize the system.

E. *FPU Enabled/Disabled*

The FPU was enable by default using the lines: FPU = -mfpu=fpv4-sp-d16 which sets up the FPU and FLOAT-ABI = -mfloat-abi=hard which sets the float point calculations to be done by hardware. By commenting out the FPU = line and changing hard to soft in the FLOAT_ABI = line, the MCU is instructed to do the float point handling through software.

F. *Complire Optimisations*

Section Incomplete

G. *Scaling in Fixed-point Arithmetic*

By changing the max_iter array from task 2 to a scale array, it is very easy to adapt the code for this task. Instead of using the defined SCALE from before, the code uses a for loop to iterate over values defined in an array. Outputting the results as usual.

H. *Power Measurement*

Section Incomplete

IV. Results and Discussions

The code was able to adequately complete 6 of the 8 tasks.

A. *Optimising Prac 1B Code and Porting to F4 Platform*

This task showed the higher processing power of the F4 platform.

B. *Adding a Second Array for MAX_ITERs*

This task showed that max iterations significantly increased the checksum value but came at the cost of a comparable increase in the time taken.

C. *Advancing Timing*

This task showed that the timing using DWT made very little difference in the accuracy of the time measurements. However, it also showed interesting stats regarding the difference in throughput.

D. *Scalability Testing*

This task showed that while very splitting the images had very little effect on the processing time of the images, it allowed the processors to compute images at a resolution that wouldn't have been possible due to memory issues.

E. *FPU Enabled/Disabled*

This task showed the effectiveness of FPU being enabled but that its effectiveness is limited to single point floats and not doubles. This is due to the F4 platform having single point float FPU.

F. *Complire Optimisations*

Section Incomplete

G. *Scaling in Fixed-point Arithmetic*

This task showed that lowering the scale value in Fixed-point calculation decreased the accuracy of the checksum value while only saving a few milliseconds.

H. *Power Measurement*

Section Incomplete

V. Conclusion

In conclusion, this practical demonstrated that while the F4 platform is significantly more powerful than the F0 platform, if used incorrectly, there is diminished value. Only when implementing the platform correctly and utilising the FPU for single point float can you really speed up the processing time from 485450ms for a 256x256 image on F0 vs 77ms on F4.

VI. AI Clause

Artificial Intelligence tools were used to assist in structuring and formatting this report, as well as to generate descriptive explanations of the code methodology. All code was written manually after AI tools aided in providing a plan to start developing the code. The code was tested for correctness and then put through ChatGPT to refine the code and add necessary comments. AI support was used in drafting the report in a concise and professional manor. AI tools proved highly effective at optimising the code. Furthermore, these tools helped greatly by creating a useful starting point for completing the report and later checking for formatting, spelling and grammatical errors.

References

[1] STMicroelectronics, “PM0214: STM32 Cortex-M4 Programming Manual,” 2023. [Online]. Available: https://www.st.com/resource/en/programming_manual/pm0214-stm32-cortexm4-mcus-and-mpus-programming-manual-stmicroelectronics.pdf

Abstract

Task 1

Resolution	Checksum (Fixed)	Time (ms) (Fixed)	Checksum (Double)	Time (ms) (Double)	Checksum (Python)	Time (ms) (Python)
128x128	429346	1456	429384	2645	429384	87
160x160	669809	2272	669829	4236	669829	142
192x192	966227	3278	966024	6093	966024	198
224x224	1315085	4462	1314999	8331	1314999	283
256x256	1715815	5822	1715812	10598	1715812	359

Task 2 F0

Max_iter	100		250		500		750		1000	
Resolution	Checksum (Fixed)	Time (ms) (Fixed)	Checksum (Fixed)	Time (ms) (Fixed)	Checksum (Fixed)	Time (ms) (Fixed)	Checksum (Fixed)	Time (ms) (Fixed)	Checksum (Fixed)	Time (ms) (Fixed)
128x128	429346	21874	968024	48764	1858360	93198	2745724	137482	3631724	181695
160x160	669809	34169	1510353	76225	2899477	145712	4281267	214830	5660801	283834
192x192	966227	49327	2180459	110117	4184222	210418	6180826	310353	8174642	410146
224x224	1315085	67163	2963856	149757	5685832	286090	8399750	422011	11110549	557772
256x256	1715815	87696	3873054	195829	7434783	374326	10985970	552290	14531940	729987

Task 2 F4

Max_iter	100		250		500		750		1000	
Resolution	Checksum (Fixed)	Time (ms) (Fixed)	Checksum (Fixed)	Time (ms) (Fixed)	Checksum (Fixed)	Time (ms) (Fixed)	Checksum (Fixed)	Time (ms) (Fixed)	Checksum (Fixed)	Time (ms) (Fixed)
128x128	429346	1456	968024	3225	1858360	6149	2745724	9063	3631724	11973
160x160	669809	2272	1510353	5033	2899477	9596	4281267	14135	5660801	18667
192x192	966227	3278	2180459	7268	4184222	13851	6180826	20410	8174642	26960
224x224	1315085	4462	2963856	9880	5685832	18823	8399750	27739	11110549	36645
256x256	1715815	5822	3873054	12911	7434783	24613	10985970	36281	14531940	47932

Task 3 F0

Resolution	Checksum (Fixed)	Time (ms) (Fixed)	CPU Cycles	Throughput
128x128	429346	21874	1049952000	749.01709
160x160	669809	34169	1640112000	749.217102
192x192	966227	49327	2367696000	747.339172
224x224	1315085	67163	3223824000	747.078003
256x256	1715815	87696	4209408000	747.308899

Task 3 F4

Resolution	Checksum (Fixed)	Time (ms) (Fixed)	CPU Cycles	Throughput
128x128	429346	1456	174790741	11248.1934
160x160	669809	2272	272728812	11263.9375
192x192	966227	3278	393453876	11243.1982
224x224	1315085	4462	535541064	11243.0596
256x256	1715815	5822	698745311	11254.917

Task 4 F0

Resolution	Checksum (Fixed)	Time (ms) (Fixed)	CPU Cycles	Throughput	Tile Count
128x128	429346	21874	1049904000	749.051331	4
256x256	1715815	87697	4209456000	747.300354	16
320x240	2010009	102589	635400704	747.692688	20
480x270	3392211	173409	4028664704	747.366028	40
640x320	6030298	308546	1925306112	746.72821	60
800x450	9412365	481697	1646619520	747.357788	104
1024x576	15423595	789849	3553013632	746.755371	144
1280x720	24099487	1234442	3418641152	746.572144	240
1600x900	37639064	1928018	2350550784	746.880981	375
1920x1080	54212127	2777384	170445824	746.601868	510

Task 4 F4

Resolution	Checksum (Fixed)	Time (ms) (Fixed)	CPU Cycles	Throughput	Tile Count
128x128	429346	1456	174784401	11248.6016	1
256x256	1715815	5826	699188342	11247.7852	4
320x240	2010009	6825	819109968	11251.2363	6
480x270	3392211	11520	1382478626	11249.3604	12
640x320	6030298	20481	2457725493	11249.4258	15
800x450	9412365	31973	3836802595	11259.375	28
1024x576	15423595	52391	1992072046	35530.2812	40
1280x720	24099487	81861	1233407928	89663.7734	28
1440x810	37639064	103599	3841961499	36431.3906	40
1600x900	37639064	127862	2458541365	70285.5781	45
1920x1080	54212127	184159	624256552	398605.344	40

Task 5 NO FPU

Resolution	Checksum (Fixed)	Time (ms) (Fixed)	Checksum (Float)	Time (ms) (Float)	Checksum (Double)	Time (ms) (Double)
128x128	429346	1456	429364	1608	429384	2645
160x160	669809	2272	669834	2535	669829	4238
192x192	966227	3278	966071	3655	966024	6096
224x224	1315085	4462	1315044	4980	1314999	8334
256x256	1715815	5822	1715808	6443	1715812	10600

Task 5 FPU

Resolution	Checksum (Fixed)	Time (ms) (Fixed)	Checksum (Float)	Time (ms) (Float)	Checksum (Double)	Time (ms) (Double)
128x128	429346	1456	429364	77	429384	2645
160x160	669809	2272	669834	120	669829	4235
192x192	966227	3278	966071	174	966024	6093
224x224	1315085	4462	1315044	237	1314999	8328
256x256	1715815	5822	1715808	309	1715812	10598

Task 7 F0

Scale	1000		10000		1000000	
Resolution	Checksum (Fixed)	Time (ms) (Fixed)	Checksum (Fixed)	Time (ms) (Fixed)	Checksum (Fixed)	Time (ms) (Fixed)
128x128	431301	19420	429278	21540	429282	22881
160x160	672930	30321	670751	33678	669809	35751
192x192	969627	43821	966561	48673	966322	51724
224x224	1321191	59544	1314881	66066	1315161	70271
256x256	1723602	77710	1716199	86271	1715788	91766

Task 7 F4

Scale	1000		10000		1000000	
Resolution	Checksum (Fixed)	Time (ms) (Fixed)	Checksum (Fixed)	Time (ms) (Fixed)	Checksum (Fixed)	Time (ms) (Fixed)
128x128	431301	1572	429278	1573	429282	1602
160x160	672930	2490	670751	2499	669809	2554
192x192	969627	3677	966561	3687	966322	3758
224x224	1321191	4889	1314881	4900	1315161	5014
256x256	1723602	6283	1716199	6289	1715788	6406

Code (Task 4 F4)

```
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * @file           : main.c
  * @brief          : Main program body
  ******************************************************************************
  * @attention
  *
  * Copyright (c) 2025 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *
  ******************************************************************************
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include <stdint.h>
#include <stdbool.h>
#include "stm32f4xx.h"
/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */
typedef struct {
    uint16_t width;
    uint16_t height;
    uint32_t execution_time_ms;
    uint64_t cpu_cycles;
    uint64_t cycles_per_ms;
    float throughput_pixels_per_sec;
    uint64_t checksum;
    bool used_tiling;
    uint16_t tile_count;
} ScalabilityResult;
/* USER CODE END PTD */
```

```
/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */
#define MAX_ITER 100
#define SCALE 1000000

// STM32F4 memory constraints - more generous than F0
#define AVAILABLE_RAM_BYTES 131072  // ~128KB available (192KB total - stack/heap)
#define MAX_DIRECT_PIXELS 32768     // Allow larger direct processing on F4
#define TILE_SIZE 128               // Larger tiles for better efficiency

// Clock frequency for STM32F4 (120MHz)
#define SYSTEM_CLOCK_FREQ 120000000

// Enhanced scalability test image sizes for F4
#define NUM_SCALABILITY_TESTS 11
/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/

/* USER CODE BEGIN PV */
// Timing variables
volatile uint32_t start_time = 0;
volatile uint32_t end_time = 0;
volatile uint32_t execution_time_ms = 0;
volatile uint64_t start_cycles = 0;
volatile uint64_t end_cycles = 0;
volatile uint64_t cpu_cycles = 0;
volatile uint64_t cylces_per_ms = 0;
volatile float throughput_pixels_per_sec = 0.0f;
volatile uint64_t checksum = 0;

// Enhanced scalability test configurations for STM32F4
const uint16_t scalability_widths[NUM_SCALABILITY_TESTS] = {
    128, 256, 320, 480, 640, 800,    // Direct processing range
    1024, 1280, 1440, 1600, // Tiled processing
    1920 // Full HD variants
};

const uint16_t scalability_heights[NUM_SCALABILITY_TESTS] = {
    128, 256, 240, 270, 360, 450,    // Direct processing range
    576, 720, 810, 900,      // Tiled processing
    1080  // Full HD variants
};

// Results storage
ScalabilityResult scalability_results[NUM_SCALABILITY_TESTS];
uint8_t current_test_index = 0;

/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
```

```
/* USER CODE BEGIN PFP */
// Core Mandelbrot functions
uint64_t calculate_mandelbrot_direct(int width, int height, int max_iterations);
uint64_t calculate_mandelbrot_tiled(int width, int height, int max_iterations, uint16_t
*tile_count);

// Tile processing functions
uint64_t process_mandelbrot_tile(int start_x, int start_y, int tile_width, int
tile_height,
                                 int full_width, int full_height, int max_iterations);

// Memory management
bool can_process_directly(int width, int height);
uint16_t calculate_tile_size(int width, int height);

// Timing functions
void init_timing_system(void);
void start_precise_timing(void);
void stop_precise_timing(void);
void calculate_throughput(int width, int height);

// Test execution
void run_scalability_test(void);
void execute_single_scalability_test(uint8_t test_index);

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{

  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */
```

```
  /* Initialize all configured peripherals */
  MX_GPIO_Init();

  /* USER CODE BEGIN 2 */
  // Initialize timing system
  init_timing_system();

  // Initialize results
  current_test_index = 0;
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    // Visual indicator: Turn on LED0 to signal scalability testing start
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);

    // Run complete scalability test suite
    run_scalability_test();

    // Turn off all LEDs
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_RESET);

    // Wait before next test cycle
    HAL_Delay(10000);

  }
  /* USER CODE END 3 */
}

/**
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Configure the main internal regulator output voltage
  */
  __HAL_RCC_PWR_CLK_ENABLE();
  __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE3);

  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
  RCC_OscInitStruct.HSEState = RCC_HSE_ON;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLM = 15;
```

```
  RCC_OscInitStruct.PLL.PLLN = 144;
  RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
  RCC_OscInitStruct.PLL.PLLQ = 2;
  RCC_OscInitStruct.PLL.PLLR = 2;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_3) != HAL_OK)
  {
    Error_Handler();
  }
}

/**
  * @brief GPIO Initialization Function
  * @param None
  * @retval None
  */
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOC_CLK_ENABLE();
  __HAL_RCC_GPIOH_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3
                         |GPIO_PIN_4|GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7,
GPIO_PIN_RESET);

  /*Configure GPIO pins : PB0 PB1 PB2 PB3 PB4 PB5 PB6 PB7 */
  GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3
                        |GPIO_PIN_4|GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
}

/* USER CODE BEGIN 4 */

// Initialize timing system
void init_timing_system(void) {
  // STM32F0 (Cortex-M0) typically doesn't have DWT
  // Use SysTick as a backup method for more precise timing
```

```
  #ifdef DWT
  // If DWT is available on this particular F0 variant
  CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
  DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
  DWT->CYCCNT = 0;
  #endif

  // SysTick is already configured by HAL_Init(), running at 1kHz
  // Can be used for microsecond-level timing if needed
}

// Start precise timing measurement
void start_precise_timing(void) {
  #ifdef DWT
  // Use DWT if available
  DWT->CYCCNT = 0;
  start_cycles = DWT->CYCCNT;
  #else
  // Fallback: estimate cycles from execution time
  start_cycles = 0;
  #endif

  start_time = HAL_GetTick();
}

// Stop precise timing measurement
void stop_precise_timing(void) {
  end_time = HAL_GetTick();
  execution_time_ms = end_time - start_time;

  #ifdef DWT
  // Use DWT if available
  end_cycles = DWT->CYCCNT;
  cpu_cycles = end_cycles - start_cycles;
  #else
  // Fallback: estimate cycles from execution time
  // Convert milliseconds to cycles: time_ms * (clock_freq / 1000)
  cpu_cycles = execution_time_ms * (SYSTEM_CLOCK_FREQ / 1000);
  #endif
}

// Calculate throughput in pixels per second
void calculate_throughput(int width, int height) {
  uint32_t total_pixels = width * height;

  if (cpu_cycles > 0) {
    float execution_time_seconds = (float)cpu_cycles / (float)SYSTEM_CLOCK_FREQ;
    throughput_pixels_per_sec = (float)total_pixels / execution_time_seconds;
  } else if (execution_time_ms > 0) {
    throughput_pixels_per_sec = (float)total_pixels * 1000.0f /
(float)execution_time_ms;
  } else {
    throughput_pixels_per_sec = 0.0f;
  }
}

// Memory management: Check if image can be processed directly
```

```
bool can_process_directly(int width, int height) {
  uint32_t total_pixels = width * height;

  // STM32F4 has more RAM, so we can handle larger images directly
  // Consider memory needed for local variables and call stack
  return (total_pixels <= MAX_DIRECT_PIXELS);
}

// Calculate optimal tile size based on available memory and image size
uint16_t calculate_tile_size(int width, int height) {
  if (can_process_directly(width, height)) {
    return 0; // No tiling needed
  }

  // For STM32F4, we can use larger tiles for better efficiency
  // Adaptive tile sizing based on image dimensions
  if (width >= 1920 && height >= 1080) {
    return 256; // Larger tiles for Full HD
  } else if (width >= 1280 || height >= 720) {
    return 192; // Medium tiles for HD content
  } else {
    return TILE_SIZE; // Default tile size
  }
}

// Direct Mandelbrot processing (for smaller images)
uint64_t calculate_mandelbrot_direct(int width, int height, int max_iterations) {
  uint64_t mandelbrot_sum = 0;

  for (int y = 0; y < height; y++) {
    // y0 = (y/height)*2.0 - 1.0  (scaled)
    int64_t y0 = ((int64_t)y * 2000000 / height) - 1000000;

    for (int x = 0; x < width; x++) {
      // x0 = (x/width)*3.5 - 2.5  (scaled)
      int64_t x0 = ((int64_t)x * 3500000 / width) - 2500000;

      int64_t xi = 0, yi = 0;
      int iteration = 0;

      while (iteration < max_iterations) {
        // xi*xi + yi*yi <= 4
        int64_t xi2 = (xi * xi) / SCALE;
        int64_t yi2 = (yi * yi) / SCALE;
        if (xi2 + yi2 > 4000000) break;

        int64_t tmp = xi2 - yi2 + x0;
        yi = (2 * xi * yi) / SCALE + y0;
        xi = tmp;
        iteration++;
      }
      mandelbrot_sum += iteration;
    }
  }
  return mandelbrot_sum;

}
```

```
// Process a single tile of the image
uint64_t process_mandelbrot_tile(int start_x, int start_y, int tile_width, int
tile_height,
                                 int full_width, int full_height, int max_iterations) {
  uint64_t tile_sum = 0;

  for (int y = start_y; y < start_y + tile_height && y < full_height; y++) {
    int64_t y0 = ((int64_t)y * 2000000 / full_height) - 1000000;

    for (int x = start_x; x < start_x + tile_width && x < full_width; x++) {
      int64_t x0 = ((int64_t)x * 3500000 / full_width) - 2500000;

      int64_t xi = 0, yi = 0;
      int iteration = 0;

      while (iteration < max_iterations) {
        int64_t xi2 = (xi * xi) / SCALE;
        int64_t yi2 = (yi * yi) / SCALE;
        if (xi2 + yi2 > 4000000) break;

        int64_t tmp = xi2 - yi2 + x0;
        yi = (2 * xi * yi) / SCALE + y0;
        xi = tmp;
        iteration++;
      }
      tile_sum += iteration;
    }
  }
  return tile_sum;
}

// Tiled Mandelbrot processing (for larger images)
uint64_t calculate_mandelbrot_tiled(int width, int height, int max_iterations, uint16_t
*tile_count) {
  uint64_t total_sum = 0;
  uint16_t tile_size = calculate_tile_size(width, height);
  uint16_t tiles_processed = 0;

  if (tile_size == 0) {
    // Should use direct processing
    *tile_count = 1;
    return calculate_mandelbrot_direct(width, height, max_iterations);
  }

  // Process image in tiles
  for (int tile_y = 0; tile_y < height; tile_y += tile_size) {
    for (int tile_x = 0; tile_x < width; tile_x += tile_size) {
      int current_tile_width = (tile_x + tile_size > width) ? width - tile_x :
tile_size;
      int current_tile_height = (tile_y + tile_size > height) ? height - tile_y :
tile_size;

      uint64_t tile_result = process_mandelbrot_tile(tile_x, tile_y,
                                                     current_tile_width,
current_tile_height,
                                                     width, height, max_iterations);
      total_sum += tile_result;
      tiles_processed++;
```

```
      // Less frequent delays needed on STM32F4 due to better performance
      if (tiles_processed % 16 == 0) {
        HAL_Delay(1);
      }
    }
  }

  *tile_count = tiles_processed;
  return total_sum;
}

// Execute a single scalability test
void execute_single_scalability_test(uint8_t test_index) {
  if (test_index >= NUM_SCALABILITY_TESTS) return;

  uint16_t width = scalability_widths[test_index];
  uint16_t height = scalability_heights[test_index];
  uint16_t tile_count = 0;

  // Store test parameters
  scalability_results[test_index].width = width;
  scalability_results[test_index].height = height;

  // Determine processing method
  bool use_tiling = !can_process_directly(width, height);
  scalability_results[test_index].used_tiling = use_tiling;

  // Start timing
  start_precise_timing();

  // Execute Mandelbrot calculation
  if (use_tiling) {
    checksum = calculate_mandelbrot_tiled(width, height, MAX_ITER, &tile_count);
  } else {
    checksum = calculate_mandelbrot_direct(width, height, MAX_ITER);
    tile_count = 1;
  }

  // Stop timing
  stop_precise_timing();
  calculate_throughput(width, height);

  // Store results
  scalability_results[test_index].execution_time_ms = execution_time_ms;
  scalability_results[test_index].cpu_cycles = cpu_cycles;
  scalability_results[test_index].cycles_per_ms = cpu_cycles/execution_time_ms;
  scalability_results[test_index].throughput_pixels_per_sec =
throughput_pixels_per_sec;
  scalability_results[test_index].checksum = checksum;
  scalability_results[test_index].tile_count = tile_count;
}

// Run complete scalability test suite
void run_scalability_test(void) {
  for (uint8_t i = 0; i < NUM_SCALABILITY_TESTS; i++) {

    execute_single_scalability_test(i);
```

```
    // Visual indicator: Turn on LED1 to signal completion
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_SET);

    // Keep LED1 ON for 2 seconds
    HAL_Delay(2000);

  }

  current_test_index = NUM_SCALABILITY_TESTS; // Mark completion
}

/* USER CODE END 4 */

/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  __disable_irq();
  while (1)
  {
  }
}

#ifdef USE_FULL_ASSERT
void assert_failed(uint8_t *file, uint32_t line)
{
}
#endif /* USE_FULL_ASSERT */
```